

PRÁCTICA 4

Interrupciones: el pulsador, SysTick y TIM2

DOCUMENTO	ASIGNATURA	TITULACIÓN	CENTRO
Del alumnado	**Arq. de Computadores II**	**Grado en Ing. Informática · 2.º**	**ESEI · Universidade de Vigo**

Contenido

1. Ficha de la práctica 3

2. Qué se construye 3

3. Los tres mecanismos, y por qué son tres 4

3.1 La palabra clave volatile 5

4. Desarrollo paso a paso 6

Paso 1. El pulsador que avisa 6

Paso 2. SysTick, y hasta dónde llega 6

Paso 3. El mismo segundo, con TIM2 7

Paso 4. El cronómetro 8

Paso 5. Confirma quién llama a tu código 9

5. Criterios de aceptación 9

6. Criterios de evaluación 10

7. Entregables 11

8. Uso de asistentes de inteligencia artificial 12

Anexo A. Lo mínimo dentro de una interrupción 13

Anexo B. Versión para PC 13

Anexo C. Conexionado 13

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesiones 6 y 7 de 13
Duración	4 horas presenciales, en dos sesiones
Modalidad	Individual, en el laboratorio
Requisitos previos	P1, P2 y P3. Se da por sabido el TM1638 y la cadencia con `HAL_GetTick()`
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · STM32CubeIDE · un reloj
Entregables	Un archivo con los cuatro proyectos, tabla de medidas y memoria (apartado 7)
Plazo	Una semana desde el final de la segunda sesión, por Moovi

2. Qué se construye

Hasta ahora todo lo que has escrito **preguntaba**: el bucle miraba una y otra vez si el pulsador había cambiado, o si ya habían pasado 100 ms. En esta sesión se da la vuelta al planteamiento: es el hardware el que **avisa**.

Se construyen cuatro programas, de menos a más:

PROYECTO	QUÉ HACE	QUÉ MECANISMO ENSEÑA
`P4_BtnInterrupcion`	El botón azul conmuta el LED verde	**EXTI**: un pin avisa al cambiar

PROYECTO	QUÉ HACE	QUÉ MECANISMO ENSEÑA
`P4_SYSTickInterrupcion`	El LED parpadea una vez por segundo	**SysTick**, el temporizador del núcleo
`P4_TIM2Interrupcion`	Lo mismo, con otro mecanismo	**TIM2**, un temporizador de periférico
`P4_Reloj`	Cronómetro completo con el TM1638	Los tres, trabajando juntos

> Fíjate en que los tres primeros tienen el **bucle principal vacío**. Todo ocurre en las funciones de atención a la interrupción. No es un truco de la práctica: es lo normal en un sistema empotrado bien hecho, y es justo lo contrario de lo que se hacía en P2 y P3.

3. Los tres mecanismos, y por qué son tres

MECANISMO	QUIÉN DISPARA	PERIODO	CUÁNDO CONVIENE
EXTI	Un pin cambia de estado	El que imponga el mundo exterior	Reaccionar a un pulsador o a un sensor sin encuestarlo.
SysTick	El propio núcleo Cortex-M	Corto y fijo	La base de tiempo del sistema. Es la que ya usa la HAL.
TIMx	Un periférico del microcontrolador	Muy amplio, gracias al prescalador	Periodos largos, frecuencias concretas y PWM.

La tercera columna es la que más importa y la que se descubre midiendo en el Paso 2. **SysTick no tiene prescalador**; los temporizadores de periférico sí. Esa diferencia decide cuál se puede usar para qué.

3.1 La palabra clave volatile

Toda variable que escriba una interrupción y lea el bucle principal —o al revés— tiene que declararse `volatile`:

LO QUE CRUZA LA FRONTERA

```
volatile uint32_t milisegundos = 0;
volatile uint32_t segundos = 0;
```

> Sin `volatile`, el compilador puede guardar la variable en un registro y no releerla nunca de memoria: el bucle esperaría un cambio que **sí ocurrió** pero que no ve. El programa compila, no avisa de nada y se queda colgado. Es el error más repetido de toda la asignatura.

`volatile` **no** hace la variable atómica ni la protege de accesos simultáneos: solo obliga a releerla. Para variables de 8, 16 o 32 bits en un Cortex-M4 el acceso ya es atómico por construcción, así que aquí basta.

4. Desarrollo paso a paso

Paso 1. El pulsador que avisa

Proyecto `P4_BtnInterrupcion`. Configura **PC13** en modo `GPIO_EXTI13` y activa su interrupción en la pestaña **NVIC**. El código va en el callback, no en el bucle:

LA FUNCIÓN DE ATENCIÓN

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13) {
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    }
}
```

Deja el `while(1)` vacío, compila y pruébalo. Pulsa el botón unas cuantas veces seguidas y **fíjate bien en lo que pasa**: es la primera cuestión de la memoria.

Paso 2. SysTick, y hasta dónde llega

Proyecto `P4_SYSTickInterrupcion`. El objetivo es que el LED conmute **una vez por segundo**. Empieza por lo que parece obvio:

EL INTENTO DIRECTO

```
/* Un segundo = tantas cuentas como ciclos de reloj hay en un segundo */
uint32_t resultado = HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq());
```

Guarda el valor de retorno en una variable y míralo con el depurador antes de seguir. Luego comprueba en la placa si el LED parpadea como esperabas.

> Si no funciona, no lo arregles a ciegas. Busca en el manual de referencia del Cortex-M4 **cuántos bits tiene el registro de recarga de SysTick**, calcula qué periodo máximo permite a la frecuencia a la que va tu placa, y de ahí sale tanto el porqué del fallo como la forma de resolverlo. Es la segunda cuestión de la memoria.

Cuando lo tengas, configura SysTick a un periodo que **sí** quepa y compón el segundo contando interrupciones dentro del callback:

COMPONER EL SEGUNDO

```
void HAL_SYSTICK_Callback(void)
{
    milisegundos++;
    if (milisegundos >= SYSTICK_POR_SEGUNDO) {
        milisegundos = 0;
        segundos++;
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    }
}
```

> **Cuidado con una trampa:** definir `HAL_SYSTICK_Callback` no basta para que se ejecute. Alguien tiene que llamar a `HAL_SYSTICK_IRQHandler()` desde `SysTick_Handler`, en `stm32f4xx_it.c`. Si falta esa línea tu función es código muerto, y **el compilador no avisa**: la función existe, simplemente no la llama nadie.

Paso 3. El mismo segundo, con TIM2

Proyecto `P4_TIM2Interrupcion`. Configura TIM2 desde el `.ioc` para que interrumpa **una vez por segundo** y conmute el LED en `HAL_TIM_PeriodElapsedCallback`. Arráncalo con `HAL_TIM_Base_Start_IT`, no con `HAL_TIM_Base_Start`: sin `_IT` el temporizador cuenta pero no interrumpe.

Para elegir el prescalador y el periodo necesitas saber **a qué frecuencia le llega el reloj a TIM2**. No es la del núcleo: TIM2 cuelga del bus APB1. Mira la pestaña *Clock Configuration* y anota el número.

> El callback es común a **todos** los temporizadores, así que hay que comprobar cuál ha sido: `if (htim->Instance == TIM2)`. Sin esa comprobación, el día que añadas un TIM3 el programa hará el trabajo dos veces por periodo.

Compara este paso con el anterior: **el mismo segundo, dos mecanismos, y solo uno de los dos podía darlo directamente.**

Paso 4. El cronómetro

Proyecto `P4_Reloj`. Junta lo anterior en una aplicación con el TM1638:

- TIM2 marca la base de tiempo e incrementa el contador **dentro de la interrupción**.
- El bucle principal solo hace dos cosas: pintar el display y leer las teclas.
- Las teclas del módulo controlan el cronómetro:

TECLA	ACCIÓN
S1	Poner el contador a cero
S2	Parar
S3	Arrancar
S4	Contar hacia adelante
S5	Contar hacia atrás

> En cuenta atrás, al llegar a cero el cronómetro **se detiene solo**. Ese es el «fin de temporización» que convierte un contador en algo útil: un temporizador de cocina, un semáforo, un ciclo de lavado.

Paso 5. Confirma quién llama a tu código

Con interrupciones aparece una duda nueva: cuando una función se ejecuta, ¿quién la ha llamado? La pila de llamadas lo responde.

1. Pon un punto de ruptura dentro de la rutina de atención del pulsador.
2. Púlsalo. Cuando el programa se detenga, mira la **pila de llamadas**.
3. Repite con el callback del temporizador y compara las dos pilas.

Verás que ninguna de las dos viene de tu bucle principal: las dos aparecen colgando de la entrada de interrupción. Es la comprobación directa de que ese código **no lo llama nadie de los tuyos**.

> Comprueba también algo del capítulo 11: mientras estás parado dentro de una rutina de atención, **el contador de milisegundos no avanza**. Párate ahí un rato, anota el valor de `HAL_GetTick()`, continúa y vuelve a pararte. Explica en la memoria lo que has visto.

5. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Pulsar el botón azul con el bucle principal vacío.	El LED conmuta. La interrupción funciona sin que nadie encueste el pin.
CA-2	Comprobar el valor devuelto por el intento directo de configurar SysTick.	Se sabe **cuánto vale y por qué**, y está justificado con el tamaño del registro de recarga.
CA-3	Cronometrar el parpadeo de `P4_SYSTickInterrupcion` durante un minuto.	Sesenta conmutaciones, con menos de **1 s de diferencia** frente a un reloj externo.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-4	Lo mismo con `P4_TIM2Interrupcion`.	Igual de exacto, y el alumno sabe decir de dónde sale el reloj de TIM2.
CA-5	En `P4_Reloj`, arrancar, parar, poner a cero y cambiar de sentido con las teclas.	Las cinco teclas hacen lo suyo y el display responde sin retrasos apreciables.
CA-6	Poner la cuenta atrás cerca de cero y dejarla llegar.	Se detiene **sola** en 00:00 y no sigue contando en negativo.

> CA-2 no pide que el programa funcione: pide **entender por qué el primer intento no funcionaba**. Es el contenido más valioso de la sesión y se evalúa aunque el proyecto acabe bien.

6. Criterios de evaluación

> **Condición previa.** Los cuatro proyectos deben compilar **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Interrupción externa	CA-1: EXTI configurada desde el `.ioc` y atendida en el callback.	Funciona, pero el pin se lee además en el bucle.	1,5
2. El límite de SysTick	CA-2: se explica por qué falla el intento directo, con números.	Sabe que falla, pero no sabe decir por qué.	2,5

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
3. Bases de tiempo correctas	CA-3 y CA-4: el segundo es un segundo con los dos mecanismos.	Uno de los dos parpadea a un ritmo que no es el pedido.	2,0
4. La aplicación	CA-5 y CA-6: el cronómetro responde y la cuenta atrás se detiene sola.	Cuenta bien pero alguna tecla no responde, o no para en cero.	2,0
5. Uso de `volatile`	Está en todo lo que cruza la frontera, y sabe decir por qué.	Lo usa porque lo pone el enunciado, sin saber qué evita.	1,0
6. Memoria	Las dos cuestiones del apartado 7.	Responde una de las dos, o sin justificar.	1,0

7. Entregables

- **Los cuatro proyectos** de STM32CubeIDE comprimidos, **sin las carpetas Debug ni Release**. Nombre: ACII_P4_Apellido1_Apellido2_Nombre.zip
- **Tabla de medidas**: lo cronometrado en CA-3 y CA-4, y el valor que devolvió el intento directo de CA-2.
- **Memoria breve**, máximo dos páginas:
    - En el Paso 1, una sola pulsación a veces conmuta el LED más de una vez. Explica por qué ocurre y qué habría que hacer para evitarlo. ¿Basta con la interrupción?
    - El intento directo de configurar SysTick para un segundo no funciona. Explica con números por qué es imposible a la frecuencia de tu placa, y qué periodo máximo sí admite.

8. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques o modifiques cualquier línea.
- **No sustituir la medida.** Los tiempos de CA-3 y CA-4 los cronometras tú, con la placa delante.

Anexo A. Lo mínimo dentro de una interrupción

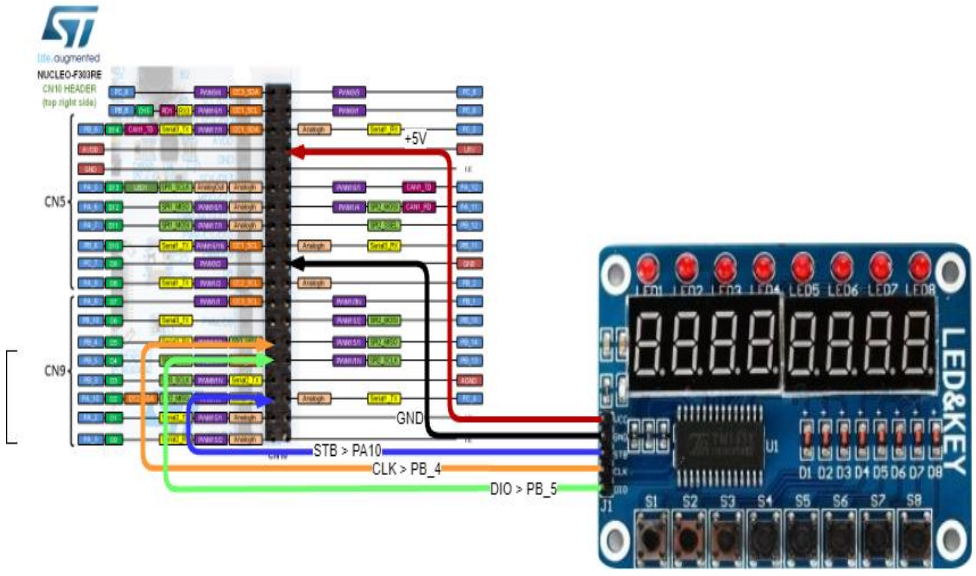
Una interrupción detiene lo que estuviera ejecutándose. Todo lo que se haga dentro se paga en latencia para el resto del sistema, así que la regla es: **contar, marcar una bandera y salir.** El trabajo de verdad se hace en el bucle principal, cuando le toca.

Por eso en `P4_Reloj` el callback solo incrementa contadores, y es el bucle el que escribe en el display. Pintar los ocho dígitos son decenas de transmisiones serie: dentro de una interrupción que se dispara mil veces por segundo, el sistema no levantaría cabeza.

Anexo B. Versión para PC

En `_simulacion_pc/reloj_interrupciones_pc.c` hay un reloj equivalente escrito para ordenador, con las interrupciones emuladas mediante señales de UNIX. Compila con `gcc` y sirve para entender el concepto sin pelearse con la configuración del microcontrolador.

Anexo C. Conexionado



CONEXIÓN DEL MÓDULO LED&KEY A LA NUCLEO-F401RE.

SEÑAL	PIN	DETALLE
LED verde LD2	PA5	Activo a 1
Botón azul B1	PC13	Activo a 0. En esta práctica, como fuente de EXTI
TM1638 · STB	PA10	Solo en `P4_Reloj`
TM1638 · CLK	PB4	
TM1638 · DIO	PB5	